

COCSC02 Data Structures

Unit-I

Unit-IA → Introduction: Basic Terminology: Elementary Data Organization, Data Structure Operations, Algorithms Complexity and Time-Space Trade off.

Unit-IB → Arrays: Array Definition and Analysis, Representation of Linear Arrays in Memory, Traversing, Insertion And Deletion in Array, Single Dimensional Arrays, Two Dimensional Arrays, Bubble Sorting, Selection Sorting, Linear Search, Binary Search, Multidimensional Arrays, Function Associated with Arrays, Character String in C, Character String Operations, Arrays as parameters, Implementing One Dimensional Array.

Unit-IA

Algorithm

Flowchart

Program

Variables

Data types: system defined (primitive) and user defined

Data structures: linear and non-linear

Abstract data types

Data Structures + Algorithms = Programs

Analysis of algorithms

Runtime

Storage

Rate of growth

Scalability

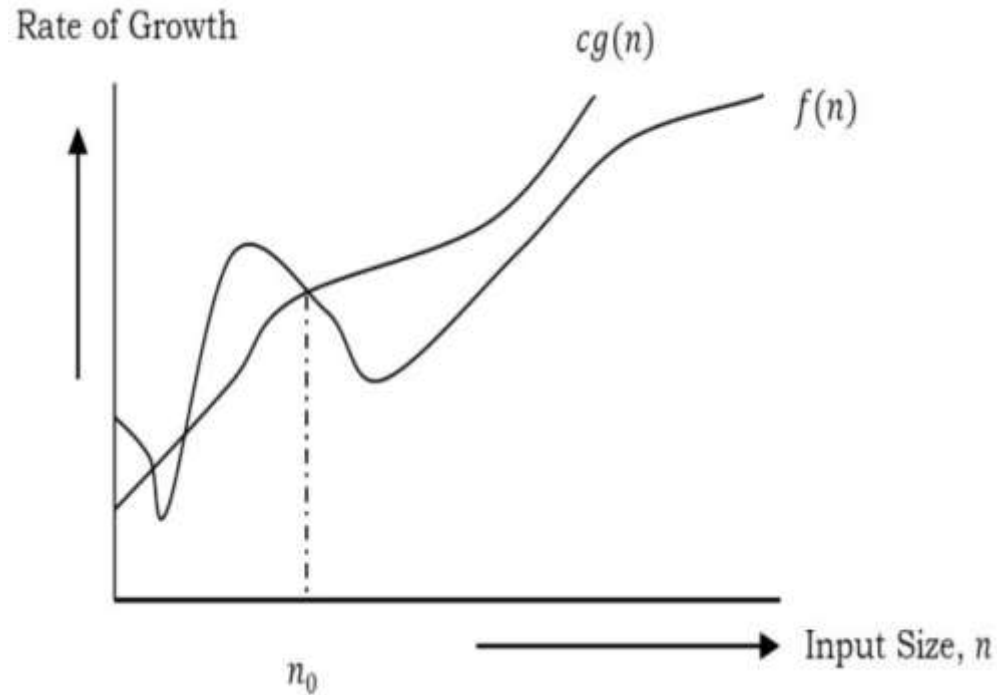
Commonly used rates of growth

Time Complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in an unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'divide-and-conquer' - Mergesort
n^2	Quadratic	Shortest path between two nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem

Type of analysis: best case, average case, worst case

Asymptotic notations: represent upper and lower bounds

Big-O Notation (Upper Bounding Function)



$O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n > n_0\}.$

$g(n)$ is an asymptotic tight upper bound for $f(n)$.

Our objective is to give the smallest rate of growth $g(n)$ which is greater than or equal to the given algorithm's rate of growth.

Example-1: Find upper bound for $f(n) = 3n + 8$

Solution: $3n + 8 \leq 4n$, for all $n \geq 8$

$\therefore 3n + 8 = O(n)$ with $c = 4$ and $n_0 = 8$

Example-2: Find upper bound for $f(n) = n^2 + 1$

Solution: $n^2 + 1 \leq 2n^2$, for all $n \geq 1$

$\therefore n^2 + 1 = O(n^2)$ with $c = 2$ and $n_0 = 1$

Example-3: Find upper bound for $f(n) = n^4 + 100n^2 + 50$

Solution: $n^4 + 100n^2 + 50 \leq 2n^4$, for all $n \geq 11$

$\therefore n^4 + 100n^2 + 50 = O(n^4)$ with $c = 2$ and $n_0 = 11$

Example-4: Find upper bound for $f(n) = 2n^3 - 2n^2$

Solution: $2n^3 - 2n^2 \leq 2n^3$, for all $n > 1$

$\therefore 2n^3 - 2n^2 = O(n^3)$ with $c = 2$ and $n_0 = 1$

Example-5: Find upper bound for $f(n) = n$

Solution: $n \leq n$, for all $n \geq 1$

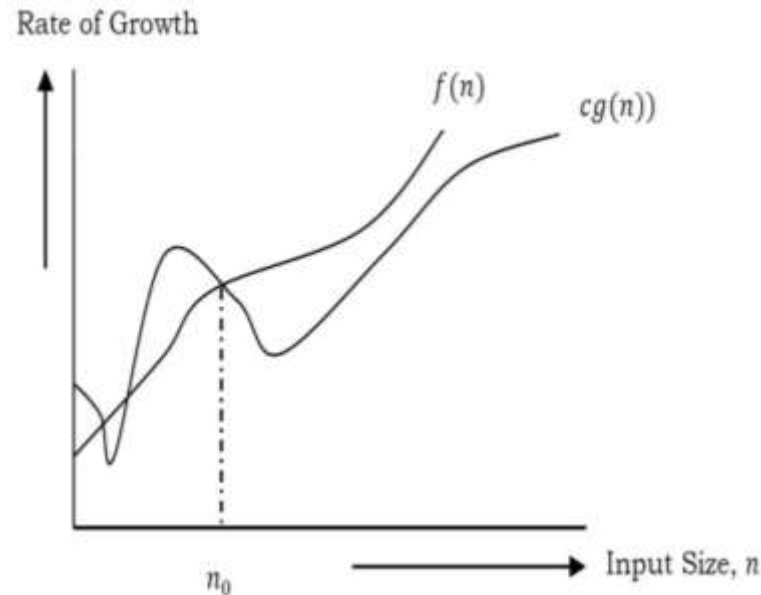
$\therefore n = O(n)$ with $c = 1$ and $n_0 = 1$

Example-6: Find upper bound for $f(n) = 410$

Solution: $410 \leq 410$, for all $n > 1$

$\therefore 410 = O(1)$ with $c = 410$ and $n_0 = 1$

Big-Ω Notation (Lower Bounding Function)



$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}.$

$g(n)$ is an asymptotic tight lower bound for $f(n)$.

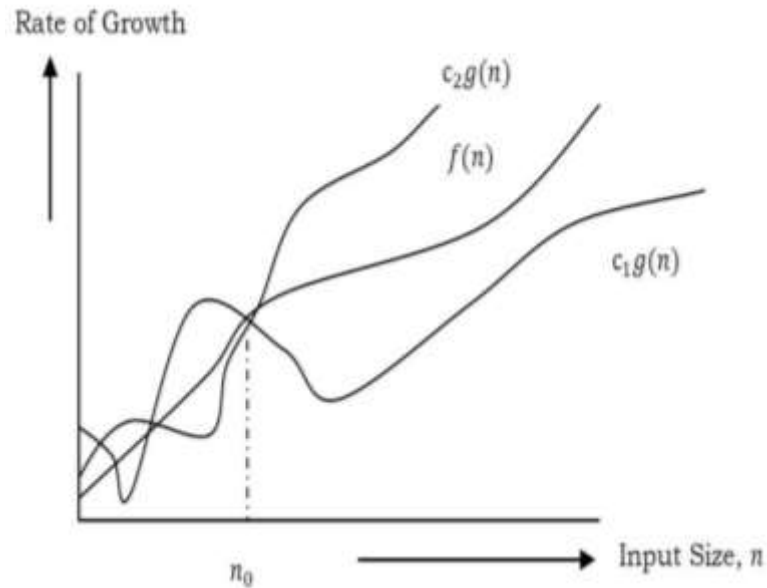
Our objective is to give the largest rate of growth $g(n)$ which is less than or equal to the given algorithm's rate of growth.

Example-1: Find lower bound for $f(n) = 5n^2$.

Solution: $0 \leq cn^2 \leq 5n^2$

$\therefore 5n^2 = \Omega(n^2)$ with $c = 5$ and $n_0 = 1$

Big- Θ Notation



$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$.

$g(n)$ is an asymptotic tight bound for $f(n)$.

$\Theta(g(n))$ is the set of functions with the same order of growth as $g(n)$.

Example 1 Find Θ bound for $f(n) = \frac{n^2}{2} - \frac{n}{2}$

Solution: $\frac{n^2}{5} \leq \frac{n^2}{2} - \frac{n}{2} \leq n^2$ for all, $n \geq 2$

$$\therefore \frac{n^2}{2} - \frac{n}{2} = \Theta(n^2) \text{ with } c_1 = 1/5, c_2 = 1 \text{ and } n_0 = 2$$

Why is it called asymptotic analysis?

For a given function $f(n)$ we are trying to find another function $g(n)$ which approximates $f(n)$ at higher values of n .

That means $g(n)$ is a curve which approximates $f(n)$ at higher values of n .

In mathematics, we call such a curve an asymptotic curve.

In other terms, $g(n)$ is the asymptotic curve for $f(n)$.

For this reason, we call algorithm analysis asymptotic analysis.

Guideline for asymptotic analysis

Loops: The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

Nested loops: Analyze from the inside out. Total running time is the product of the sizes of all the loops.

Consecutive statements: Add the time complexities of each statement.

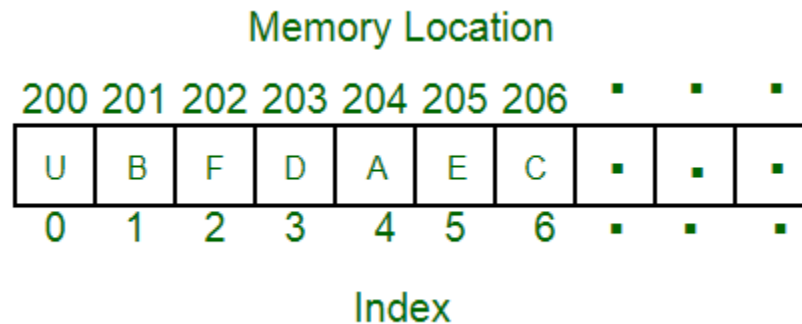
If-then-else statements: Worst-case running time: the test, plus either the then part or the else part (whichever is the larger).

Logarithmic complexity: An algorithm is $O(\log n)$ if it takes a constant time to cut the problem size by a fraction (usually by $\frac{1}{2}$).

Unit-IB

Array: linear data structure, similar elements, ordered, stored in contiguous memory locations

One-dimensional array = vector



```
int A[10];

for (int i=0; i<10; i++)
    cin>>A[i];

int *ptr ;
ptr=A;
ptr=&A[0];

for (int i=0; i<10; i++)
    cout<<*(ptr+i);

for (int i=0; i<10; i++)
    cout<<*(A+i);
```

Array operations: accessing an element, traversing, traversing backward, insertion, deletion, merging two arrays

Linear search

```
Linear_search (A, n, T)
i = 1
While i <= n
    If A[i] = T
        Return i
Return -1
```

Time complexity = $O(n)$

Binary search

Binary_search (A, n, T)

L = 1

R = n

While L <= R

 m = floor ((L + R)/2)

 If A[m] = T

 Return m

 Else if A[m] < T

 L = m+1

 Else if A[m] > T

 R = m-1

Return -1

Time complexity = $O(\log n)$

Bubble sort

Read the array left to right and compare two consecutive elements and swap them if necessary
Repeat till the array is sorted

Bubble_sort (A, n)

i=1

while i<=n-1

 j=n-1

 while j>=i

 if A[j]>A[j+1]

 swap A[j] and A[j+1]

 j=j-1

 i=i+1

Time complexity = $O(n^2)$

Selection sort

The array is divided into a sorted part and an unsorted part

Initially, the sorted part is empty

Select the smallest element in the unsorted part and swap it with the leftmost element in the unsorted part

The leftmost element of the unsorted part becomes the rightmost element of the sorted part

Repeat till the unsorted part is empty

Selection_sort (A, n)

i=1

while i<=n-1

 min=i

 j=i+1

 while j<=n

 if A[j]<A[min]

 min=j

 j=j+1

 swap A[i] and A[min]

 i=i+1

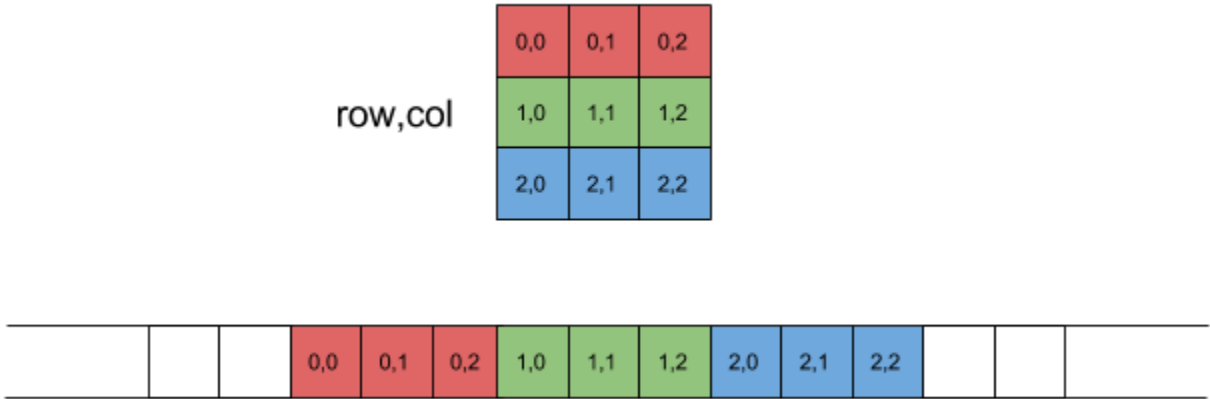
Time complexity = $O(n^2)$

Two-dimensional array = matrix

```
int A[10][20];  
A[5][8]=100;
```

C and C++ use row-major ordering

Can you calculate the address of the element (x,y)?




```
for (int i=0; i<3; i++)  
    for (int j=0; j<3; j++)  
        C[i][j]=A[i][j]+B[i][j];
```

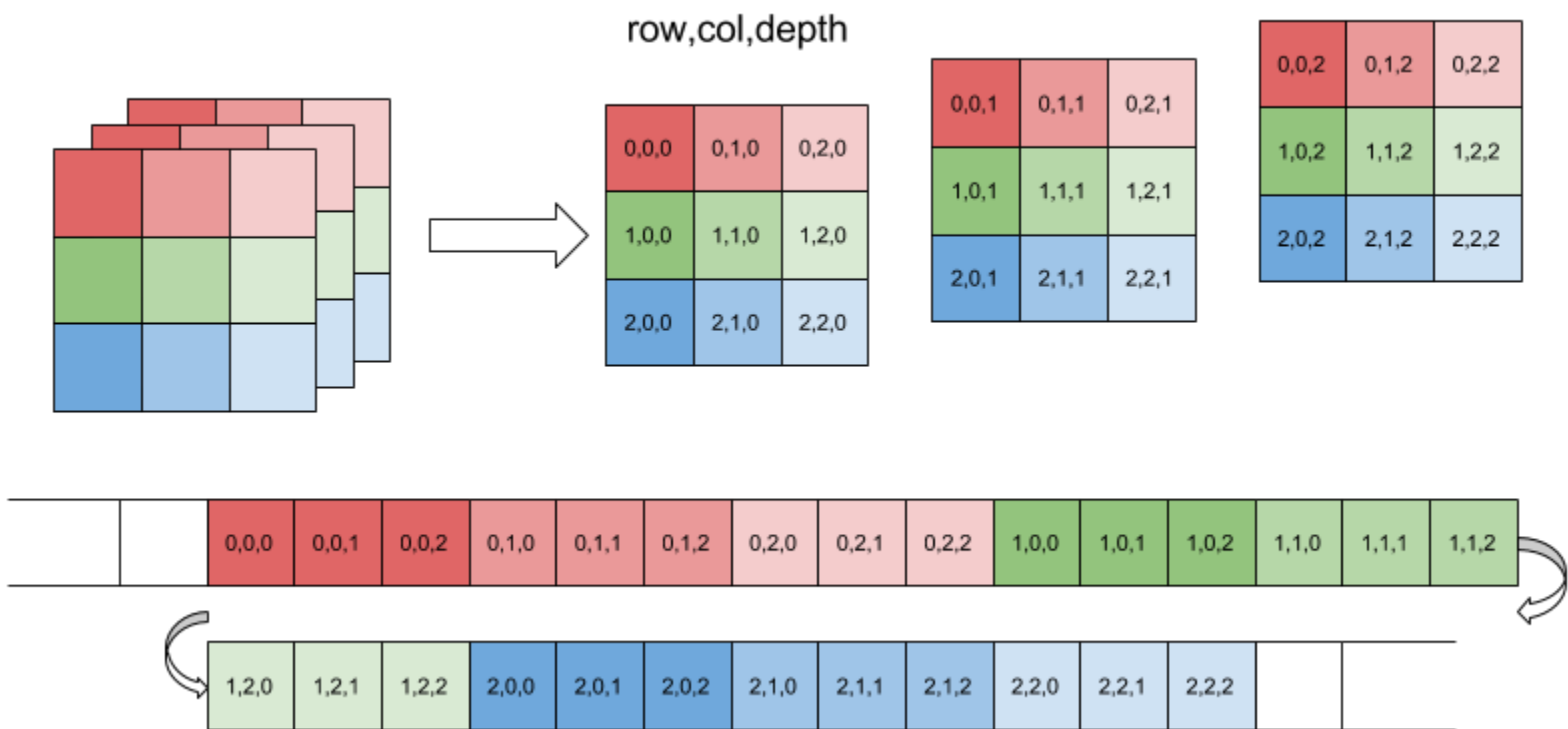
Matrix multiplication

Multi-dimensional array

```
int A[10][20][30];  
A[5][8][12]=250;
```

C and C++ use row-major ordering

Can you calculate the address of the element (x,y)?



```
//function definition
int func1 (int Array[])
{
return 0;
}
```

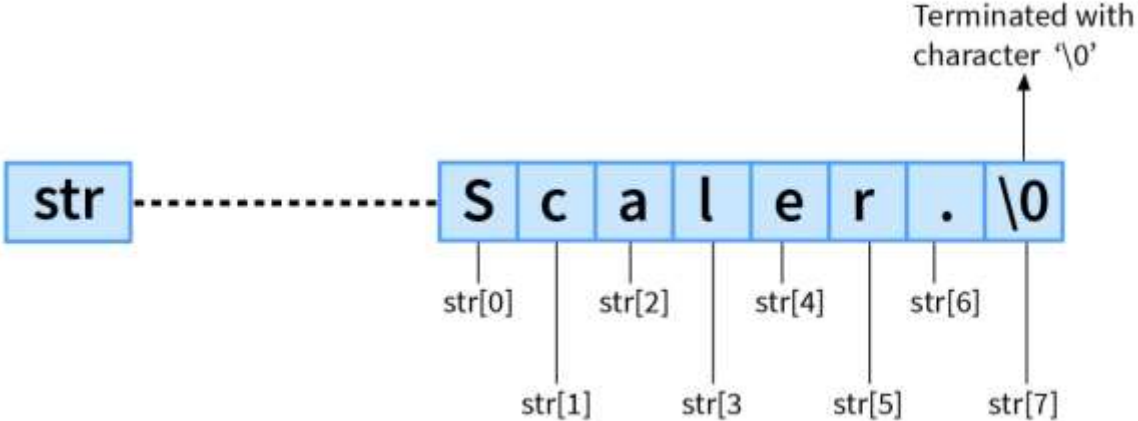
```
//function call
int A[10];
int x = func1 (A);
```

```
//function definition
int* func2 ()
{int A[10];
return A;
}
```

```
//function call
int *ptr;
ptr = func2 ();
```

Character string

```
char s1[50];  
cin>>s1;  
cout<<s1;  
gets(s1);  
puts(s1);
```



```
for (i=0; s1[i]!='\0'; i++)  
    cout<<s1[i];
```

Find length, reverse, palindrome, prefix, suffix, substring, change case

End of Unit